

Reymes Olide

Jennifer Bui

CECS 327 Sec01

May 5, 2026

Distributed File System 2

Chord Integration

The Distributed File System is built on top of a Chord based distributed Hash Table. The system consists of five peer nodes arranged in a static ring with 256 slots. It uses an 8-bit keyspace. Node IDs are manually assigned as 2,4,6,8, and 10, which are spaced across the ring to distribute the load evenly and predictably. All stored objects are addressed by SHA hash keys. When a key needs to be stored or retrieved, the hash digest is converted to an integer and reduced modulo 256 to produce a logical ring position. The routing layer then performs a successor lookup. It scans the sorted list of known peers and returns the first node whose ID is greater than or equal to the key's position. If no node exists, the routing wraps around to the lowest ID node, completing the ring topology. Each node maintains an 8 entry finger table built at startup. Entry i stores the successor. This enables logarithmic time routing in a larger network. Remote operations are also encoded as JSON payloads and transmitted over TCP sockets. When the responsible node is the local node, the request will bypass the network entirely and will interact directly with the node's on disk storage. This avoids unnecessary socket overhead.

Metadata and Page Design

The files in the DFS are represented as two separate object types, a metadata object and one or more page objects. This separation prevents any single node from becoming a bottleneck and enables parallelism during read and write operations. When a file is created with “touch”, the system creates a metadata object keyed by the SHA-1 hash of the string *metadata:<filename>*.

The metadata records the filename, total size, number of pages, a version counter, and an ordered list of page descriptors. Each page descriptor contains the page number, its hashed page key, the size in bytes, and the list of node IDs that hold replicas of that page. When data appends to a file, the content is split into fixed sized 1 KB chunks. Each chunk is wrapped in a page object keyed by the hash of *page:<filename>:<page_no>* and routed independently through the chord ring.

After all the pages are stored, the metadata object is updated to reflect the new page count, total size, and version number. Reads recreate the file by grabbing the pages in descriptor order and concatenating their content.

Distributed Sorted Strategy

The sort operation uses a map reduce style distributed algorithm that leverages the Chord ring’s numeric ordering. The sort is invoked as *sort_file(input, output)* and proceeds in three phases. In the partition phase, the input file is read from the DFS and parsed into key value pairs. Each record’s numeric key is used as a Chord ring position to determine which peer is responsible for that record’s partition bucket. Records are routed to their responsible peers and appended to a temporary JSON file that is stored in that node’s *sort_partitions* directory. In the collection phase, the client queries each peer in strict ascending order. Each peer returns its

locally sorted partition. Since the ring is numerically ordered and records were routed by key value, the partitions arrive in order. The client concatenates them to produce a global sorted result. In the commit phase, the sorted content is submitted as a single sort file paxos operation. The output file is created and populated in one committed round, so the sorted output either appears complete across the replicas or not at all.

Replication Strategy

The system uses a replication factor of $R = 3$ with the successor based placement on the Chord ring. When an object is written, the routing layer calls *get_replica_peers*, which identifies the key's primary owner index in the sorted peer list and returns the node and the next two nodes. All 3 receive independent PUT requests. Reads attempt the primary owner first and fall back to the remaining replicas only if the primary cannot be reached. Deletes are then issued to all 3 replicas. The strategy replicates both file metadata and raw page data. Since the replica set is computed deterministically from the key, any node in the ring can construct the correct set without any coordination. Page descriptors in the metadata record the assigned replica node IDs, making placement auditable.

Paxos Message Flow

To serialize replicated DFS state changes, all write operations are coordinated through a simplified Paxos Protocol. The leader is assigned to the node with the lowest ID. The Paxos replica set consists of the first 3 peer nodes, 2, 4, and 6. Every state changing operation is submitted to the leader as a typed operation object rather than executed directly. The protocol proceeds in two phases: the Accept phase and the Learn Phase. In the Accept phase, the leader increments a ballot number and broadcasts *Paxos Accept* to all 3 replicas. Each follower stores the proposed operation in memory, logs the ballot number, and replicates it. Each follower stores the proposed operation in memory, logs the ballot number, and replies with a *Learn* acknowledgement. In the Learn phase, once the leader receives acknowledgements from a majority, it broadcasts *Paxo Learn* to instruct all replicas to mark the operation as committed. The leader then applies the operation to its own DFS state. Each step is timestamped and written to *Paxos_log.txt*. The majority threshold ensures that as long as two replicas are reachable, the system can make progress. Every replica applies operations in ballot number order. This guarantees that all replicas converge to the same file system state.

Failure assumptions

The system is designed around a crash, stop failure model. Nodes may halt unexpectedly, but they won't send corrupted or malicious data. Messages can be delayed, duplicated, or reordered. The TCP layer mitigates most packet loss, but it doesn't eliminate timing issues. The system was tested under a simulated follower crash. A Paxos round was issued with a connection

error, which was then caught and logged without aborting. The two remaining replicas responded successfully and satisfied the majority rule. The operation was then committed, and the file state remained consistent. The crashed replica then splits off until it can recover. The system also cannot handle Byzantine failures, systems that isolate the leader and multiple proposals from multiple leaders at the same time.

Limitations and Future Improvements

The current Chord topology is static. Nodes are currently unable to join or leave at runtime. Adding dynamic membership would require us to implement Chord's stabilize and notify protocols. That would include key migrations when a new node joins between two existing successors. The Paxos leader is statically assigned. If the leader node crashes, no new proposals can be issued. A full Paxos implementation would include a prepare phase, allowing surviving followers to elect a new leader with a higher ballot number and resume operations. Another future improvement would be to change how Committed Paxos are held. Currently committed Paxos state is held in volatile memory. If a node restarts, it loses all the accepted and committed operation history. A write ahead log on disk would allow recovering nodes to replay missed operations and rejoin the current system state. Additionally, the DFS index is stored as a single Chord key. Concurrent *touch* or *delete_file* operations could cause lost updates to the index under a high load of concurrency. A conflict free index service would address this in a production system.